

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Course Name: Computer Networks

Course Code: CSA0706

Course Outcome Covered: CO5 – Measure network performance using key metrics with simulation tools. (BL4, BL5)

Assessment Tool Weightage: 30%

Annexure A – Pseudocode Writing Task (Algorithm Design)

I, Y.Prem kumar (Reg. No: 192212216), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Prem kumar*

Assessment Overview

This document presents pseudocode-based algorithm design solutions for three network performance monitoring scenarios, as required under Course Outcome CO5. Each solution includes a clear statement of inputs, outputs and constraints, a fully structured pseudocode algorithm with appropriate validation and control structures, a supporting flowchart, and a discussion of how the algorithm contributes to efficient, reliable network operation. All three tasks address the measurement and management of key network performance metrics — throughput, packet loss, bandwidth utilization, delay, and link quality — using simulation-style reasoning consistent with tools such as Cisco Packet Tracer.

Question 1: Network Throughput and Packet Loss Calculator

1.1 Problem Understanding

A student running a simulation in Cisco Packet Tracer has recorded the following observations, and the goal is to compute two key performance metrics — throughput and packet loss percentage — and classify the overall network condition.

- Inputs: totalBits (data received, in bits) = 8000; time (seconds) = 4; packetsSent = 200; packetsReceived = 180.
- Outputs: throughput (bits per second), packetLoss (percentage), and a network classification (Efficient / Inefficient).
- Constraints: time must not be zero and packetsSent must not be zero, since both appear as denominators and would cause a division-by-zero error.
- Classification rule: the network is Efficient only if throughput > 1500 bps AND packetLoss < 10%; otherwise it is Inefficient.

1.2 Algorithm Design

The algorithm first validates the inputs to avoid runtime errors, then computes throughput as the ratio of total bits received to the time taken, and computes packet loss as the percentage of packets that failed to arrive relative to packets sent. A conditional check then classifies the network, and a final summary is displayed. This directly matches the requirement to validate time and total packets sent, calculate both metrics, classify the network, and output a complete summary.

1.3 Pseudocode

```
ALGORITHM CalculateThroughputAndPacketLoss
BEGIN
    // ---- Step 1: Input ----
    INPUT totalBits          // total data received, in bits
    INPUT time              // time taken, in seconds
    INPUT packetsSent       // total packets transmitted
    INPUT packetsReceived   // total packets successfully received

    // ---- Step 2: Validation ----
    IF (time == 0) OR (packetsSent == 0) THEN
        DISPLAY "Error: time and packetsSent must not be zero."
        STOP
    END IF

    // ---- Step 3: Compute throughput (bps) ----
    throughput = totalBits / time

    // ---- Step 4: Compute packet loss percentage ----
    packetsLost = packetsSent - packetsReceived
    packetLoss = (packetsLost / packetsSent) * 100

    // ---- Step 5: Classify network performance ----
    IF (throughput > 1500) AND (packetLoss < 10) THEN
        status = "Efficient"
    ELSE
        status = "Inefficient"
    END IF

    // ---- Step 6: Display summary ----
    DISPLAY "----- Network Performance Summary -----"
    DISPLAY "Throughput      : ", throughput, " bps"
    DISPLAY "Packets Sent    : ", packetsSent
    DISPLAY "Packets Received: ", packetsReceived
    DISPLAY "Packet Loss     : ", packetLoss, " %"
    DISPLAY "Network Status  : ", status
END
```

1.4 Worked Example (Dry Run)

Using the given values: $\text{throughput} = 8000 / 4 = 2000$ bps. $\text{packetsLost} = 200 - 180 = 20$, so $\text{packetLoss} = (20 / 200) \times 100 = 10\%$. Since $\text{throughput} (2000) > 1500$ but $\text{packetLoss} (10\%)$ is not less than 10% , the condition fails and the network is classified as Inefficient. This shows that both conditions of the AND check must hold simultaneously for an Efficient classification.

1.5 Flowchart

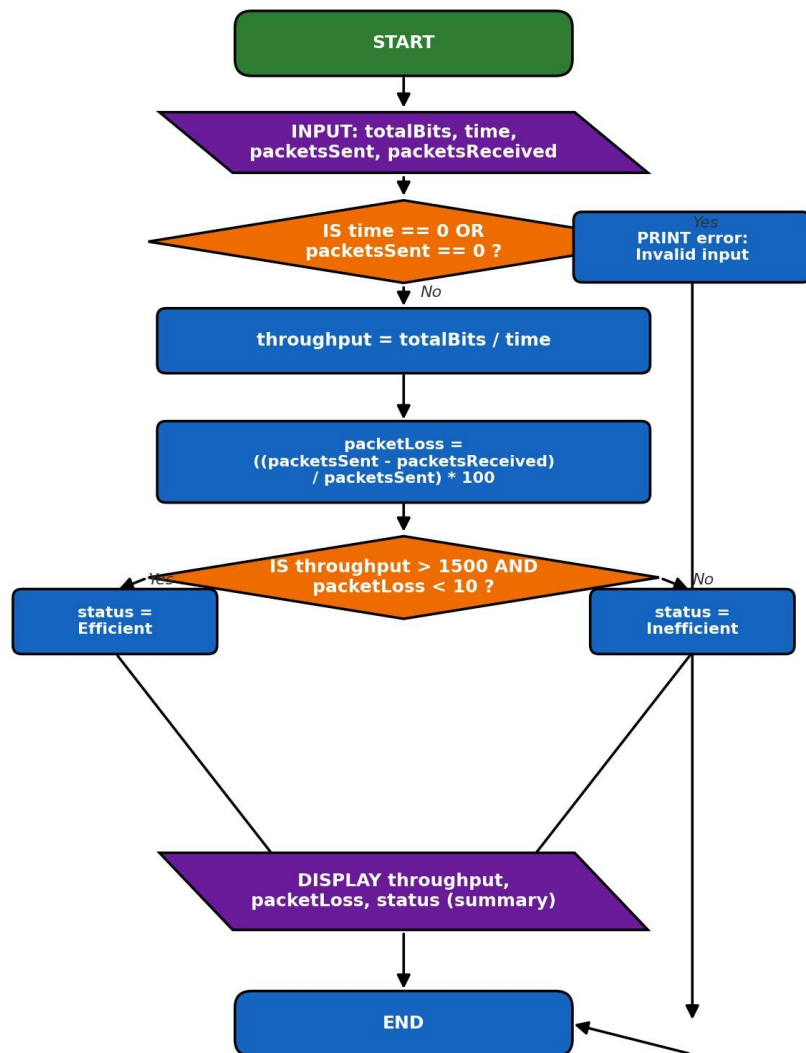


Figure 1: Flowchart for throughput and packet-loss calculation and classification

1.6 Control Structures and Completeness

- Selection (IF-THEN-ELSE) is used twice: once for input validation and once for network classification.
- A compound logical condition (AND) ensures both throughput and packet-loss thresholds are satisfied before a network is labelled Efficient.
- Defensive programming: the algorithm terminates gracefully with an error message instead of crashing when time or packetsSent is zero.
- The summary output block satisfies the requirement to present a complete, human-readable performance report rather than isolated numbers.

Question 2: Continuous Bandwidth Utilization Monitor

2.1 Problem Understanding

An organization operates multiple branch links whose bandwidth usage changes over time. The administrator needs an always-on monitoring process that detects overloaded links (above 80% utilization) and recommends failover to a backup link, so that congestion is caught and corrected before it affects service.

- Inputs: a list/array of network links, each with a known total capacity; live bandwidth-usage readings collected periodically.
- Outputs: a continuously updated utilization log, overload flags, and backup-switch recommendations/alerts.
- Constraints: monitoring must be continuous (a loop, not a one-off check) and must not stop after a single overloaded link is found — all links are checked every cycle.

2.2 Algorithm Design

The design stores each link's usage in an array/list that is refreshed every monitoring cycle inside an infinite (or longrunning) loop. A FOR loop iterates over all links each cycle, and an IF condition flags any link whose utilization exceeds the 80% threshold, triggering a recommendation to switch traffic to a backup link. This structure mirrors real network-management systems (e.g., SNMP polling) that sample interface counters at fixed intervals.

2.3 Pseudocode

ALGORITHM MonitorBandwidthUtilization

BEGIN

```
// ---- Step 1: Setup ----
DECLARE linkList[1..N]           // N network links with known capacity
DECLARE usage[1..N]             // current utilization % of each link
DECLARE THRESHOLD = 80          // overload threshold in percent
DECLARE T = monitoring interval (seconds)

// ---- Step 2: Continuous monitoring loop ----
WHILE (TRUE) DO
    FOR i = 1 TO N DO
        usage[i] = GetBandwidthUsage(linkList[i]) // sample current usage

        IF (usage[i] > THRESHOLD) THEN
            DISPLAY "WARNING: Link ", linkList[i].name, " at ",
            usage[i], "% utilization"
            IF (linkList[i].hasBackup == TRUE) THEN
                SWITCH traffic of linkList[i] to
                linkList[i].backupLink
                DISPLAY "Traffic switched to backup link."
            ELSE
                DISPLAY "ALERT: No backup available for ",
                linkList[i].name
            END IF
        ELSE
            DISPLAY linkList[i].name, " is within safe limits (",
            usage[i], "%)"
```

```

        END IF
    END FOR

    WAIT for T seconds        // pause before next sampling cycle

END WHILE
END

```

2.4 Flowchart

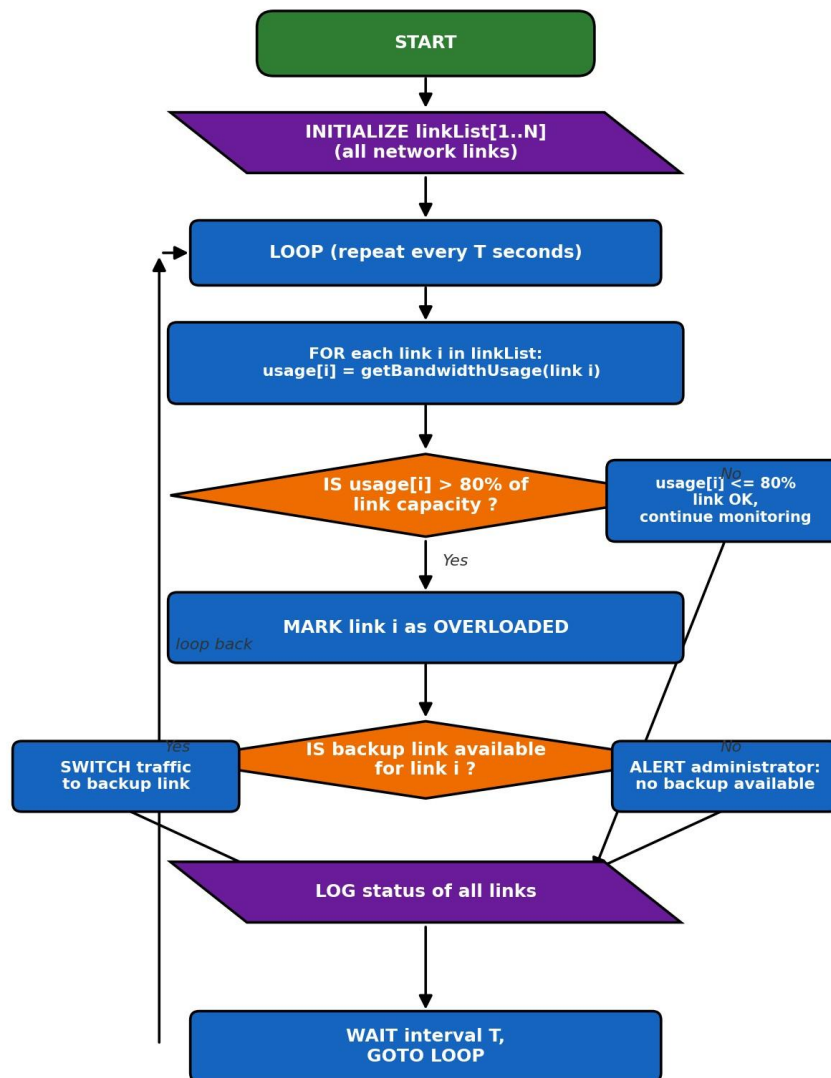


Figure 2: Flowchart for continuous bandwidth-utilization monitoring and backup switching

2.5 How This Supports Efficient Bandwidth Management

This algorithm supports efficient bandwidth management and congestion prevention in several ways:

- Early detection: sampling every link on a fixed interval means overload is detected within one cycle instead of only after users notice slowdowns.
- Automated response: switching to a backup link the moment the 80% threshold is crossed avoids the packet queuing and delay that build up as a link approaches saturation.
- Continuous operation: the WHILE(TRUE) loop with an internal FOR loop means no link is ever skipped, and monitoring persists indefinitely rather than as a single check.
- Scalability: storing links in an array/list means the same loop logic works whether the organization has 2 branches or 200, without rewriting the algorithm.
- Administrator visibility: alerts are raised only when no backup exists, which prevents alert fatigue while still guaranteeing critical failures are reported.

Question 3: Adaptive Best-Path Selection Across Branch Links

3.1 Problem Understanding

Six branch offices are connected to headquarters through multiple links whose bandwidth, delay and utilization vary throughout the day. Rather than routing purely on shortest distance (as classic distance-vector routing would), the administrator wants the path with the best overall quality, continuously re-evaluated, with automatic failover if a link degrades or fails.

- Inputs: for every link — bandwidth[i], delay[i], utilization[i] — refreshed at every monitoring interval; weighting factors for each metric; failure/overload status of each link.
- Outputs: a computed link-quality score per path, the currently selected best path to each branch, failover switches, and administrator alerts.
- Constraints: monitoring must be periodic and ongoing; a failed or overloaded link must never be selected as the best path; the algorithm must re-select automatically without manual intervention.

3.2 Algorithm Design

The algorithm stores bandwidth, delay and utilization for every link in parallel arrays/lists, refreshed inside a repeating loop. A weighted scoring function combines the three metrics into a single link-quality score — rewarding high bandwidth and penalizing high delay and high utilization. Overloaded or failed links are excluded from selection by forcing their score to a minimal value, and the link with the maximum remaining score is chosen as the best path. Any change in the selected path triggers a routing switch and, where a link has failed, an alert to the administrator.

3.3 Pseudocode

ALGORITHM SelectBestPathAdaptive

BEGIN

 // ---- Step 1: Setup ----

 DECLARE links[1..N] // N candidate links, HQ to each branch

 DECLARE bandwidth[1..N], delay[1..N], utilization[1..N]

 DECLARE score[1..N]

 DECLARE W1, W2, W3 // weights for bandwidth, delay, utilization

 DECLARE previousBestPath = NULL

 DECLARE T = monitoring interval (seconds)

 // ---- Step 2: Continuous monitoring and evaluation loop ----

 WHILE (TRUE) DO

 FOR i = 1 TO N DO

 bandwidth[i] = GetBandwidth(links[i])

 delay[i] = GetDelay(links[i])

 utilization[i] = GetUtilization(links[i])

 // ---- Step 3: Fault / overload detection ----

 IF (links[i].status == "FAILED") OR (utilization[i] > 80) THEN

 score[i] = -INFINITY // exclude this link from

 selection

 DISPLAY "ALERT: Link ", links[i].name, " is overloaded or
 down"

 ELSE


```

        // ---- Step 4: Weighted link-quality score ----
        score[i] = (W1 * bandwidth[i]) - (W2 * delay[i]) - (W3 * utilization[i])

        END IF

    END FOR

    // ---- Step 5: Select best path ----
    bestPath = links[index of MAX(score)]

    IF (bestPath != previousBestPath) THEN
        SWITCH routing table entry to bestPath
        DISPLAY "Routing updated -> new best path: ", bestPath.name
        previousBestPath = bestPath
    END IF

    DISPLAY "Current best path: ", bestPath.name, " | scores: ", score
    WAIT for T seconds
END WHILE
END

```

3.4 Flowchart

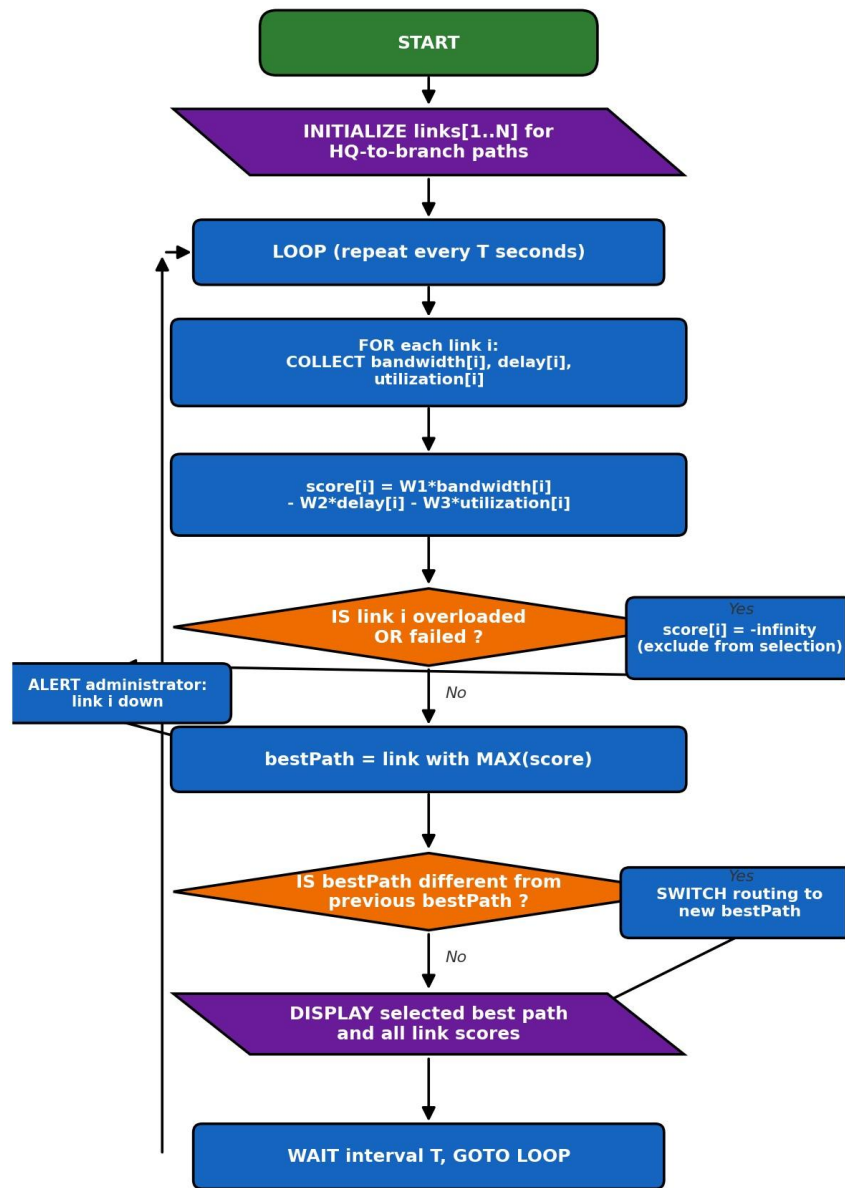


Figure 3: Flowchart for adaptive, quality-based best-path selection with fault tolerance

3.5 How This Improves Reliability, Routing Efficiency and Fault Tolerance

- Reliability: because bandwidth, delay and utilization are all re-measured every cycle, the chosen path always reflects current network conditions rather than a static, one-time measurement.
- Routing efficiency: the weighted score ($W1$, $W2$, $W3$) lets the administrator tune how much bandwidth is valued relative to delay and congestion, producing a path choice that is genuinely 'best' rather than merely 'shortest'.

- Fault tolerance: forcing score = -INFINITY for a failed or overloaded link guarantees it can never be selected as the best path, so traffic is automatically re-routed the moment a problem is detected — with no manual reconfiguration.
- Change-driven switching: routing is only updated when bestPath actually changes, which avoids unnecessary route flapping while still reacting immediately to real degradation.
- Adaptability: because the loop re-runs indefinitely at interval T, the algorithm naturally adapts to time-of-day traffic patterns (e.g., peak-hour congestion) without needing to be restarted.

Consolidated Summary

Across all three tasks, the pseudocode consistently follows a common structure: (1) declare and validate inputs, (2) use loops (FOR / WHILE) to repeat measurement over links or time, (3) use conditional logic (IF / ELSE, including compound AND conditions) to classify, detect faults, or select the best option, and (4) display a clear, complete summary of results for the network administrator or user. This consistency reflects sound algorithm-design practice: each solution is deterministic, handles edge cases such as zero inputs or failed links, and separates data collection from decision-making so the logic remains easy to follow and extend.

Self-Assessment Against Rubric

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Problem Understanding: all inputs, outputs and constraints for each scenario are explicitly listed before the algorithm is designed. Algorithm Design: each solution is logically complete, validated, and traced through a worked/dry-run example (Question 1). Use of Control Structures: FOR loops, WHILE loops, and nested IF-ELSE/compound conditions are used purposefully rather than decoratively. Pseudocode Structure: consistent indentation, step comments, and BEGIN/END blocks are used throughout. Completeness: zero-division checks, overload/failure exclusion, and change-driven switching demonstrate handling of edge cases beyond the basic happy path.